DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

LESSON PLANS

TM-40

USAREUR-AF

LESSON PLAN OUTLINES — TM-40 SPECIALIST TRACKS

USAREUR-AF Operational Data Team — C2DAO

HEADQUARTERS
UNITED STATES ARMY EUROPE AND AFRICA
(USAREUR-AF)
Wiesbaden, Germany

DRAFT — NOT FOR OFFICIAL USE. FOR TRAINING PLANNING PURPOSES ONLY.

12 MARCH 2026

DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

DRAFT

LESSON PLAN OUTLINES — TM-40 SPECIALIST TRACKS

USAREUR-AF OPERATIONAL DATA TEAM — C2DAO

Covers: TM-40G (ORSA) | TM-40H (AI Engineer) | TM-40I (MLE) | TM-40J (PM) | TM-40K (KM) | TM-40L (SWE) **Version:** 1.1 — March 2026 (updated: specialist tracks renumbered G–L; WFF tracks are TM-40A–F, prereq TM-20)

Abbreviated LP outlines for TM-40 specialist courses. Instructors at TM-40 level must have deep domain SME background — these outlines supplement SME knowledge, not replace it. Expand using `LP_TEMPLATE.md` as needed.

PART A — TM-40G: ORSA SPECIALIST

Duration: 5 days (40 hours) | **T:I ratio:** 4:1 | **Instructor req:** FA49 or equivalent; TM-40G certified or C2DAO ORSA SME

Block 1 — ORSA Role on MSS and Analytical Product Standards

Hours: 1.0 | **Method:** Brief | **Day:** 1 | **Time:** 0800–0900

Purpose: Establishes what it means to be an ORSA analyst on MSS and what the non-negotiable product standards are. Product standards (uncertainty communication, documentation, reproducibility) must be established before any modeling work.

TLO: The trainee will describe the ORSA analyst role on MSS and state the three non-negotiable product standards: uncertainty characterization, documented assumptions, and reproducibility.

Key Delivery Notes: - The ORSA role on MSS is not about building dashboards — it is about building models and delivering commander decision products. Everything else is infrastructure. - Non-negotiable standards: (1) Every estimate has a confidence interval. (2) Every model has documented assumptions. (3) All simulations are reproducible (seed set). - Foundry data model review: ORSA accesses data through Code Workspace, not Pipeline Builder.

Assessment: Evaluator will ask about each standard during the practical exercise brief.

Block 2 — Code Workspace Setup

Hours: 2.0 | **Method:** Lab | **Day:** 1 | **Time:** 0900–1100

Purpose: A broken workspace on Day 4 costs the trainee their modeling time. Set up and verify the workspace on Day 1, not later.

TLO: The trainee will configure a Python/R Code Workspace with required packages, verify Foundry dataset connectivity via Spark/pandas read, and successfully execute a test write transaction to a curated output dataset.

Key Delivery Notes: - Package installation: `pip install statsmodels scipy pandas numpy matplotlib seaborn` (Python) or `install.packages(c("forecast", "lpSolve"))` (R). Foundry workspaces may not persist installed packages across session restarts — build an install cell at the top of every notebook. - Verify connectivity: `spark.read.table("ri.foundry.main.dataset.[RID]")` or pandas via the Foundry API. Confirm schema appears. - Write transaction test: `transaction = dataset.start_transaction()` → write → `transaction.commit()`. This must work before any model outputs are written. Test it today.

Assessment: Evaluator confirms workspace is configured at start of Practical Exercise (Day 5).

Block 3 — Foundry Dataset Connectivity

Hours: 0.75 | **Method:** Lab | **Day:** 1 | **Time:** 1115–1200

Key Delivery Notes: - Three access patterns: Spark DataFrame (large datasets), pandas (smaller datasets via `.toPandas()`), direct Foundry REST API. For TM-40G, Spark → pandas conversion is the primary pattern. - Schema inspection: `df.printSchema()` or `df.dtypes`. Confirm column names and types match the Ontology definition.

Block 4 — Writing Outputs to Foundry

Hours: 2.0 | **Method:** Lab | **Day:** 1 | **Time:** 1300–1500

Purpose: Model outputs that don't return to Foundry can't be integrated with Ontology visualizations or downstream applications. The write transaction pattern must be mastered on Day 1.

Key Delivery Notes: - Transaction pattern: `with dataset.transaction() as t: t.write(df)`. The transaction must commit — an uncommitted transaction leaves no data. - Output schema: define the output schema explicitly before writing. Column names must follow C2DAO naming convention. Output dataset goes in the project's Datasets folder. - Test: after writing, open the output dataset in Foundry and confirm row count, column names, and a sample of values.

Block 5 — Data Profiling in Code Workspace

Hours: 1.75 | **Method:** Lab | **Day:** 1 | **Time:** 1515–1700

Key Delivery Notes: - Null analysis: `df.isnull().sum() / len(df)`. Columns with >30% nulls require an explicit handling decision before modeling. - Outlier detection: IQR or z-score. Flag outliers before modeling — do not silently include them. - Feature distributions: histograms for numerics, value counts for categoricals. This is the "know your data" step before any model.

Block 6 — Regression Modeling

Hours: 2.0 | **Method:** Lab | **Day:** 2 | **Time:** 0830–1030

Purpose: Regression is the foundational ORSA tool on MSS. The standard is not just "fit a model" — it is "fit, validate, and document a model that an FA49 evaluator can reproduce."

TLO: The trainee will build a linear regression model, run residual analysis and cross-validation, document assumptions and validation statistics, and write output to Foundry.

Key Delivery Notes: - Feature selection rationale must be documented — not just "I chose these because the model fit well." - Validation statistics required: R^2, RMSE, MAE, residual plot inspection, VIF for multicollinearity. - Set `random_state` before train/test split and before any stochastic operation. Reproducibility standard applies here too.

Assessment: Evaluated in Practical Exercise Tasks 2 and 6.

Block 7 — Classification Models

Hours: 1.25 | **Method:** Lab | **Day:** 2 | **Time:** 1045–1200

Key Delivery Notes: - Logistic regression and decision tree. Calibration check is required — Army ORSA products that present probabilities must be calibrated. - Cross-validation: at minimum k=5 fold. Report mean and std of validation metric. - Calibration: `sklearn.calibration.calibration_curve`. If the curve deviates substantially from the diagonal, apply Platt scaling or isotonic regression.

Block 8 — Model Validation Standards

Hours: 2.0 | **Method:** Lab | **Day:** 2 | **Time:** 1300–1500

DRAFT

Key Delivery Notes: - Residual analysis for regression: residuals vs. fitted plot, QQ plot for normality. Flag if heteroscedasticity or non-normality is present — document the finding. - USAREUR-AF validation standard: every model must have a documented model validation section before it is presented to a commander.

Block 9 — Practice Build (Regression to Foundry Output)

Hours: 1.75 | **Method:** Lab | **Day:** 2 | **Time:** 1515–1700

Key Delivery Notes: Second dataset, full cycle: profile → regression → validate → write to Foundry → build Quiver visualization of output. No instructor assistance.

Block 10 — Time Series: Stationarity and Model Identification

Hours: 2.0 | **Method:** Lab | **Day:** 3 | **Time:** 0830–1030

TLO: The trainee will test a time series for stationarity, plot ACF/PACF, identify plausible ARIMA order parameters, and document the model selection rationale.

Key Delivery Notes: - ADF test for stationarity. If non-stationary: first difference and retest. - ACF/PACF plots: trainees must explain what they see in the plots, not just run the code. "I see a cutoff at lag 2 in the PACF, suggesting AR(2)" — this is the level of explanation the evaluator expects. - Document the selection: "Selected ARIMA(2,1,1) based on: ADF test requiring 1 difference (d=1), PACF cutoff at 2 suggesting p=2, ACF gradual decay suggesting q≤1."

Block 11 — ARIMA/SARIMA Build

Hours: 1.25 | **Method:** Lab | **Day:** 3 | **Time:** 1045–1200

Key Delivery Notes: - 90% confidence intervals are required — not optional. The evaluator will look for them. - Out-of-sample forecast: at minimum 6 periods forward. Plot with historical data and confidence bounds.

Block 12 — Monte Carlo Simulation

Hours: 2.0 | **Method:** Lab | **Day:** 3 | **Time:** 1300–1500

TLO: Given a COA comparison scenario, the trainee will build a Monte Carlo simulation with ≥1,000 trials, distribution selection rationale, seed set for reproducibility, and probability at defined operational thresholds.

DRAFT

Key Delivery Notes: - Minimum 1,000 trials. 100 trials produce unstable distributions — the evaluator knows this and will test for it. - Distribution selection: `np.random.normal()`, `np.random.triangular()`, `np.random.uniform()` — justify the choice from the data or from operational knowledge. - Probability at threshold: "probability of <80% readiness at D+30" — count trials below threshold / total trials. - Set `np.random.seed()` before any stochastic operations.

Block 13 — Sensitivity Analysis and Logistics Risk

Hours: 1.75 | **Method:** Lab | **Day:** 3 | **Time:** 1515–1700

Key Delivery Notes: - Sensitivity analysis: vary each input parameter ±10% and observe output change. Identify which inputs drive the most output variance. - Logistics stockage risk: Poisson distribution for demand arrivals is a common model. Show the application to Class IX stockage.

Blocks 14–17 — Optimization, Wargame Data, Quiver/Contour

Hours: 7.5 total | **Days:** Day 4

Block 14 (LP): `scipy.optimize.linprog` for resource allocation. Document constraints explicitly. Sensitivity analysis on binding constraints.

Block 15 (Scheduling Optimization): Formulate maintenance scheduling as LP or ILP. Operational constraints from the mission calendar.

Block 16 (Wargame Data Architecture): Collection template design. Aggregation pipeline for exercise data. Post-exercise analysis pipeline.

Block 17 (Quiver/Contour for ORSA): Forecast dashboard with uncertainty bounds. COA comparison visualization. The uncertainty bounds must be visible — not buried.

Blocks 18–21 — Commander Brief Standards and Practical Exercise

Day: 5

Block 18–19 (Communicating Uncertainty): The brief posture is the hardest skill. Translate quantitative output to operational language. Three prohibitions: (1) point estimates without bounds, (2) unqualified predictions ("will"), (3) methods-paper language.

Block 20 (Scenario Brief): Provide scenario. 45 minutes planning time before build.

Block 21 (Practical Exercise): 4-hour build: regression + time series forecast + commander brief from provided readiness dataset.

Go standard: Pass 5 of 6 tasks. Commander brief reviewed by FA49 evaluator. Brief without uncertainty characterization fails that element — hard standard.

PART B — TM-40H: AI ENGINEER

Duration: 5 days (40 hours) | **T:I ratio:** 4:1 | **Instructor req:** AIP Logic authoring experience; C2DAO AI SME designation

MANDATORY: Block 1 (AI Safety Seminar, 2.0 hours) cannot be skipped, rescheduled, or made up independently.

Block 1 — AI Safety Seminar (MANDATORY)

Hours: 2.0 | **Method:** Seminar | **Day:** 1 | **Time:** 0800–1000

Purpose: The most consequential block in the course. AI systems that produce incorrect outputs affect operational decisions. Trainees who treat this as a compliance checkbox build workflows that fail in production in ways that matter.

TLO: The trainee will complete an AIP Authorization Checklist for a sample workflow, identify at least 5 prohibited use cases by category, and describe the human-in-the-loop requirement for Ontology write operations.

Key Delivery Notes: - Cover the Army CIO AI policy (April 2024) — prohibited use categories, human oversight requirements, OPSEC implications of AI outputs. - Operational context: AI outputs may be used in briefings, decision products, and Ontology records that commanders rely on. An incorrect extraction in a SITREP summary is a readiness reporting error. - Human-in-the-loop: NO AIP Logic workflow may write to production Ontology Objects without a tested human checkpoint. This is a non-negotiable design requirement, not a policy preference. - Walk through TM-40H Appendix B (Prohibited Use Cases) line by line.

Assessment: AIP Authorization Checklist completion is evaluated in Practical Exercise Task 6.

Block 2 — AIP Platform Architecture

Hours: 1.75 | **Method:** Lecture | **Day:** 1 | **Time:** 1015–1200

Key Delivery Notes: - Full stack: AIP Logic (workflow authoring), Agent Studio (agent orchestration), Code Workspaces (Python transforms), LLM endpoints (model API). - AIP Logic is the workflow layer — it orchestrates the other components. Understanding the architecture prevents the common error of trying to do everything in the prompt.

Blocks 3–4 — AIP Logic: First Workflow and Conditional Chains

Hours: 3.75 | **Method:** Lab | **Day:** 1 | **Time:** 1300–1700

Block 3 TLO: Author a first AIP Logic workflow with a prompt template, input configuration, output binding, and run a test — observing the structured JSON output.

Block 4 TLO: Build conditional chains with looping and error handling — workflows that behave differently based on intermediate outputs.

Key Delivery Notes (combined): - Prompt template: explicit context for military terminology. LLMs don't know what "DODAAC" or "MTOE" mean without definition. - Structured JSON output: require the workflow to output JSON, not prose. This is what enables Ontology write integration. - Error handling: configure what happens when the LLM fails to produce structured output. Routes to human review queue, not silent failure.

Blocks 5–8 — Advanced AIP Logic and Python Transforms

Day: 2

Block 5: Multi-step chains with parallel branches and Ontology write Actions. **Block 6:** Python transforms — extracting and formatting Ontology data for AIP context. Military terminology definitions must be explicit. **Block 7:** Context management — chunking strategies for large datasets that exceed context windows. **Block 8:** Ontology write integration with human review queue. All objects begin as `status = Draft`. **This design is non-negotiable.**

Blocks 9–12 — RAG Architecture

Day: 3

Block 9: Semantic search setup, retrieval from Ontology Objects, context construction. **Block 10:** Full RAG pipeline: retrieval → context formatting → LLM → JSON output → Ontology write with review gate. **Block 11:** RAG quality evaluation: retrieval relevance scoring, grounding failure detection, hallucination patterns on operational data. **Block 12:** End-to-end workflow practice with instructor coaching.

Blocks 13–16 — Agent Studio and Evaluation

Day: 4

Block 13: Agent Studio architecture, tool definition, tool-use authorization (what can the agent do?), memory scope. **Block 14:** Testing Agent Studio — confirming authorization controls prevent out-of-scope tool use. **Block 15:** Red-teaming: adversarial prompt testing. For every workflow, ask: "How could this produce a dangerous or misleading output?" **Block 16:** AI Output Validation Framework; complete AIP Authorization Checklist for a practice workflow.

Blocks 17–20 — Deployment and Practical Exercise

Day: 5

Block 17: Production deployment: pipeline scheduling, monitoring, failure alerting, rollback procedures. **Block 18–19:** Scenario brief; planning time; authorization checklist guidance. **Block 20:** Practical Exercise (4 hours). Build an AIP Logic workflow extracting structured SITREP data, routing to human review queue, writing to Ontology via Action.

Hard No-Go: Workflow writes to production Objects without human checkpoint.

PART C — TM-40I: ML ENGINEER

Duration: 5 days (40 hours) | **T:I ratio:** 4:1 | **Instructor req:** ML production experience; C2DAO MLE SME designation

Block 1 — MLE Role and Model Governance Overview

Hours: 1.0 | **Method:** Brief | **Day:** 1

Purpose: Read TM-40I Chapter 9 (governance) first. The governance destination makes the technical work purposeful. Every model must eventually produce a model card.

TLO: The trainee will describe the MLE role on MSS and state the required components of a model governance document (model card).

DRAFT

Blocks 2–5 — Workspace Setup, Foundry Write Pattern, Feature Engineering

Day: 1

Block 2 (GPU Workspace Setup): GPU allocation, package management, Foundry connectivity. Write transaction test on Day 1 — this must work before modeling begins.

Block 3 (Foundry Write Pattern): Transaction-based output writes. Test this on Day 1. Trainees who skip this lose evaluation time on Day 5.

Block 4 (Feature Engineering Principles — Lecture): Null handling strategies (impute vs. drop vs. sentinel), encoding (one-hot vs. ordinal), scaling (standard vs. min-max), leakage detection.

Block 5 (Feature Engineering Practice): Apply to provided dataset. Document each feature decision in writing.

Blocks 6–9 — Feature Pipeline Build

Day: 2

Block 6: Full feature pipeline: raw → feature matrix. Every step documented. **Block 7:** Leakage audit. Can any feature be derived from the label? Run the procedure from TM-40I Ch 3. This is evaluated. **Block 8:** Output to Foundry curated dataset via write transaction. Verify schema. **Block 9:** Experiment setup: train/test split, baseline model (majority class), cross-validation configuration.

Blocks 10–13 — Model Training and Evaluation

Day: 3

Block 10: Training: scikit-learn or PyTorch in Code Workspace. Cross-validation. Hyperparameter tuning. **Block 11:** Evaluation: accuracy, precision/recall, ROC-AUC against USAREUR-AF thresholds. Calibration check is required. **Block 12:** Model comparison — train two models, select winner, document rationale including trade-offs. **Block 13:** Experiment tracking in Foundry model registry. Version each model.

Blocks 14–17 — Deployment and MLOps

Day: 4

Block 14: Serving endpoint deployment. Latency and throughput baseline. **Block 15:** Connect to Ontology via Actions that invoke inference and write predictions to Object properties. **Block 16:** Monitoring pipeline: data drift detection (PSI or KS test), threshold definition from validation set baseline,

DRAFT

alert routing. **Block 17:** Operational patterns: readiness prediction, logistics demand forecasting, anomaly detection.

Blocks 18–21 — Governance and Practical Exercise

Day: 5

Block 18: Model governance document (model card) completion. Hard requirement: assumptions, training data, limitations, intended use restrictions, responsible AI declaration. **Block 19:** Deployment approval workflow; C2DAO governance for deployed models. **Block 20:** Scenario brief, planning time. **Block 21:** Practical Exercise (4 hours). Feature pipeline → train → evaluate → deploy → monitoring → governance doc.

Hard No-Go: Model calibration not performed. Governance document missing required sections. Drift pipeline fails to detect evaluator-seeded drift event.

PART D — TM-40J: PROGRAM MANAGER

Duration: 3 days (24 hours) | **T:I ratio:** 6:1 | **Instructor req:** Program management background; TM-30 certified; GFEBS/IMS proficiency

---

Block 1 — Program Data Architecture

Hours: 1.5 | **Method:** Lab | **Day:** 1

Purpose: The four Object Types (Program, Milestone, Resource, Risk) must be designed on paper before touching Ontology Manager. A poorly designed PM data model means rebuilding — which takes a day the course doesn't have.

Key Delivery Notes: - Design exercise on paper first: 4 Object Types, their properties, and the Link Types connecting them. 15 minutes on paper. Then build. - Program is the parent Object. Milestones, Resources, and Risks all link back to Program. - The data-as-of timestamp is not a column — it is a pattern. Every pipeline output that feeds a PM dashboard must include a computed `data_as_of_date` column using `CURRENT_DATE()`.

Block 2 — Ontology: PM Object Types, Link Types, Actions

Hours: 1.75 | **Method:** Lab | **Day:** 1

Key Delivery Notes: - Link Types: `Program → Milestone` (ONE_TO_MANY), `Program → Risk` (ONE_TO_MANY). Create both. - Action configuration: `UpdateMilestoneStatus` Action with parameters for `new_status` (GREEN/AMBER/RED) and `comments`. Access restricted to Editor role.

Block 3 — IMS Pipeline

Hours: 2.0 | **Method:** Lab | **Day:** 1

Purpose: The IMS pipeline is the heart of the PM data product. DATEDIFF for milestone variance and the RAG status computed column are the key deliverables.

Key Delivery Notes: - IMS Excel exports have dates as text. CAST before DATEDIFF — this is the most common pipeline failure. - DATEDIFF: `variance_days = DATEDIFF('day', planned_completion, actual_completion)`. Positive = late. Negative = ahead. - RAG status: `CASE WHEN variance_days > 30 THEN 'RED' WHEN variance_days > 0 THEN 'AMBER' ELSE 'GREEN' END AS milestone_rag`. - Data-as-of timestamp: `CURRENT_DATE() AS data_as_of_date`. Add to every pipeline output.

Block 4 — GFEBS Obligation Pipeline

Hours: 2.0 | **Method:** Lab | **Day:** 1

Key Delivery Notes: - Configure in Append mode before the first run. Add `CURRENT_DATE() AS snapshot_date`. Run twice. Verify two distinct snapshot dates in the output.

Block 5 — Workshop Milestone Dashboard

Hours: 2.0 | **Method:** Lab | **Day:** 2

Key Delivery Notes: - RAG conditional formatting: table rows colored by `milestone_rag`. Configure RED, AMBER, GREEN color rules. - Data-as-of widget: a text or metric widget displaying the most recent `data_as_of_date` value. This is a hard No-Go if absent on the evaluation.

Blocks 6–9 — Quiver, Contour, Reporting, Checklist

Day: 2

DRAFT

Block 6 (Quiver): Obligation rate chart with quarterly target reference line. Q2 target = 50%. **Block 7 (Contour):** Portfolio health matrix. Sort by `overall_status` ascending (RED at top). **Block 8 (Reporting):** Scheduled pipeline refresh. Build failure notification. PDF export procedure. **Block 9 (PM Dashboard Standards Checklist):** Walk every item. This is the evaluator's checklist.

Blocks 10–11 — Practice Run and Practical Exercise

Day: 3

Block 10 (Practice Run, 3.25 hrs — UNGRADED): Full stack from a different dataset. Instructor coaching available. This is instruction, not optional review.

Block 11 (Practical Exercise, 4 hrs): Full stack from provided IMS and GFEBS data. 7 tasks. Go standard: 6 of 7. Hard No-Go: dashboard missing data-as-of timestamp.

PART E — TM-40K: KNOWLEDGE MANAGER

Duration: 3 days (24 hours) | **T:I ratio:** 6:1 | **Instructor req:** KM background; TM-30 certified; AIP Logic configuration proficiency

---

Block 1 — KM Role on MSS

Hours: 1.0 | **Method:** Brief | **Day:** 1

Key Delivery Notes: - The operational problem: institutional knowledge walks out at every PCS cycle. MSS is the solution — structured capture systems that survive personnel turbulence. - Design principle: "Build for the person who replaces you in 12 months." Every KM system must be operable by someone who was not there when it was built.

Block 2 — Knowledge Object Types

Hours: 2.0 | **Method:** Lab | **Day:** 1

Key Delivery Notes: - Five Object Types: Document, Lesson, AAR, SOP, ExpertiseProfile. Design on paper first. - Link Types: `Lesson → AAR` (Lesson is extracted from AARs), `Lesson → Unit` (which unit generated the lesson), `SOP → Unit` (which unit owns it).

Block 3 — AAR Submission Form

Hours: 0.75 | **Method:** Lab | **Day:** 1

Key Delivery Notes: - Required fields: unit, date, event type, location, what happened (text), key lesson, classification marking. - Field validation: required fields must produce an error if empty. Confirm the validation fires before submission.

Block 4 — Lessons Learned Pipeline

Hours: 2.0 | **Method:** Lab | **Day:** 1

Key Delivery Notes: - Tagging taxonomy: unit, event type (exercise, operation, transition), echelon, TTP category, classification level. Design the taxonomy before building the pipeline. - Distribution routing: `IF classification = 'SECRET', route to classified-only queue`. Routing logic depends on the tagging.

Block 5 — AIP Logic: Summarization with Human Review Gate

Hours: 1.75 | **Method:** Lab | **Day:** 1

Purpose: AIP summarization turns a 10-page AAR into a 3-paragraph lesson summary. The human review gate turns a draft into a vetted lesson.

Key Delivery Notes: - Workflow output: AIP-generated lessons ALWAYS begin with `status = Draft`. A Draft lesson requires KM review before `status = Published`. No exceptions. - The evaluator will specifically attempt to trigger a workflow that writes directly to Published status. Design the workflow so this is impossible by construction.

Blocks 6–10 — Knowledge Browser, Version Control, Expertise, PCS Transfer, Prompt Iteration

Day: 2

Block 6 (Knowledge Browser): Search by keyword, filter by tag/unit/date. Drill-down from search result to lesson full text.

Block 7 (SOP Version Control): Version tagging. Review notification workflow when SOP review date is passed.

Block 8 (ExpertiseProfile + Privacy Act): Skills taxonomy. Privacy Act authorities (TM-40K Section 8-1) apply to skills databases — the evaluator will ask about this.

Block 9 (PCS Transfer): Key person dependency analysis. Transfer package must name specific Foundry projects, Object Types, pipelines, data quality status, and required contacts. Generic template = No-Go.

Block 10 (AIP Prompt Iteration Lab): Test against 5 provided documents. Score extraction quality. Revise prompt. Retest. Repeat. The prompt matters more than the workflow configuration.

Blocks 11–13 — PCS Package Review and Practical Exercise

Day: 3

Block 11 (PCS Draft Review, 2.25 hrs): Each trainee presents draft PCS package to instructor. Instructor reviews against TM-40K Ch 9 completeness criteria. Revise based on feedback. **This block is instruction, not optional preparation.**

Block 12: Practical exercise scenario brief.

Block 13 (Practical Exercise, 4 hrs): 6 tasks: Ontology design, AAR form, lessons pipeline, AIP workflow with review gate, knowledge browser, PCS transfer package.

Hard No-Go: AIP workflow auto-publishes without human review gate.

PART F — TM-40L: SOFTWARE ENGINEER

Duration: 5 days (40 hours) | **T:I ratio:** 4:1 | **Instructor req:** OSDK/Platform SDK experience; TM-40L certified; C2DAO SWE SME designation

---

Block 1 — SWE Role and OSDK Architecture

Hours: 1.0 | **Method:** Lecture | **Day:** 1

Purpose: TM-40L SWEs are building production software on Foundry. Security must be established as a first principle before any code is written. Read TM-40L Chapter 9 (Security) before Day 1.

Key Delivery Notes: - The 5-layer data stack: raw data → pipelines → Ontology → OSDK → applications. SWEs live at layer 4–5. - OSDK architecture vs. REST: OSDK is type-generated from the Ontology definition. It is not a generic REST client. Type generation means changes to the Ontology

break the OSDK client — trainees must understand this dependency. - Security principle established on Day 1: NO hardcoded credentials, tokens, or secrets in code. EVER. This is the hardest No-Go in the course.

Blocks 2–5 — OSDK Fundamentals

Day: 1

Block 2 (OSDK Setup): Auth architecture, client initialization, token handling. First single-object query. Verify authentication before anything else.

Block 3 (Filtering and Sorting): Query predicates, sort expressions, field selection. Build on Block 2's working client.

Block 4 (Pagination with ResourceIterator): This is an evaluated skill. If you write a query that retrieves page 1 and stops, the evaluator will use an Object set requiring 2+ pages. Use `ResourceIterator` and iterate all results. Every time.

Block 5 (Link Traversal): Query related Objects across Link Types. Join-like patterns without leaving the Ontology layer.

Blocks 6–9 — OSDK Advanced

Day: 2

Block 6 (Action Execution): Async response pattern — OSDK Action execution returns a task ID, not a result. Poll for completion with task ID.

Block 7 (Error Handling and Retry): Action execution failures, timeout patterns, structured error response.

Block 8 (Object Subscriptions): Real-time change notifications via WebSocket. Connecting subscriptions to application state for live-updating UIs.

Block 9 (Bulk Operations): Batch queries, bulk Action submissions. The pattern: never loop over Objects making individual API calls. Use bulk patterns. The evaluator tests with 200+ Objects.

Blocks 10–13 — Platform SDK and TypeScript FOO

Day: 3

Block 10 (Platform SDK): Dataset read/write operations, write transactions, file resources, branch management. Different layer than OSDK — Platform SDK operates on datasets, not Ontology Objects.

Block 11 (Platform SDK Exercise): Build a dataset integration using the read/write transaction pattern.

Block 12 (TypeScript FOO): Repository structure, computed property implementation, function registration. FOO extends the Ontology with computed properties that Foundry evaluates.

Block 13 (FOO Bulk Query Patterns): N+1 queries are the most common FOO performance failure. If your FOO makes 1 API call per Object, it will time out at 200+ Objects. Use bulk query patterns from TM-40L Chapter 5.

Blocks 14–17 — Action Validators and Slate

Day: 4

Block 14 (Action Validators): Multi-condition validation with cross-field logic (`if status = DEPLOYED, location must be non-null`). Clear error message for each validation condition.

Block 15 (Validator Testing): Build a test suite with minimum 8 cases (4 valid, 4 invalid). Each test case paired with expected error message. The evaluator runs this test suite against the trainee's actual code — all 8 must pass.

Block 16 (Slate Applications): Application structure, Foundry API integration, widget binding, initial data load.

Block 17 (Slate State Management): When a user triggers an Action that updates an Object, the UI must refresh. Configure state variable tied to Action completion, not a timer. This is the hardest Slate skill.

Blocks 18–21 — CI/CD, Security, and Practical Exercise

Day: 5

Block 18 (CI/CD Lecture): PR workflow, automated testing integration, C2DAO deployment checklist. Code review is non-negotiable before any deployment.

Block 19 (Security Lecture): Token handling (Foundry Secrets API, not hardcoded), input sanitization (injection risk on all user inputs), OPSEC (no architecture details in commit messages).

Block 20 (Scenario Brief): Provide scenario; planning time.

Block 21 (Practical Exercise, 4 hrs): OSDK paginated query → Action validator with test suite → TypeScript FOO → Slate application with error states → deployment checklist.

Hard No-Go: Any hardcoded credential in application code. Validator test suite not fully passing (all 8 cases required).

USAREUR-AF Operational Data Team — UNCLASSIFIED TM-40 Specialist Lesson Plan Outlines | Version 1.1 | March 2026

DRAFT